

Deep Learning for NLP – Homework 2

Student name: *Georgios Karteris*
sdi: 2100258

Course: *Artificial Intelligence II (M138, M226, M262, M325)*
Semester: *Spring Semester 2025*

Contents

1	Abstract	2
2	Data Processing and Analysis	2
2.1	Pre-processing	2
2.2	Analysis	2
2.3	Data Partitioning for Train, Test, and Validation	3
2.4	Vectorization	3
3	Algorithms and Experiments	3
3.1	Experiments	3
3.1.1	Table of Trials	4
3.2	Hyper-parameter Tuning	5
3.3	Optimization techniques	5
3.4	Evaluation	5
3.4.1	ROC curve	5
3.4.2	Learning Curve	6
3.4.3	Confusion matrix	6
4	Results and Overall Analysis	8
4.1	Results Analysis	8
4.2	Comparison with the first project	8
4.3	Comparison with the second project	9
5	Bibliography	9

1. Abstract

This project involves performing sentiment analysis on tweets, aiming to classify them as either positive or negative. This is a binary classification problem that requires a combination of natural language processing (NLP) techniques and machine learning algorithms. To tackle this problem, we employed a systematic approach that included data pre-processing, and fine-tuning using BERT and DistilBERT models. The process involved multiple stages of experimentation, including baseline model evaluation, various pre-processing techniques, and fine-tuning to optimize the models' performance. The highest score reached was 0.85522 for BERT and 0.84857 for DistilBERT.

- BERT notebook: <https://www.kaggle.com/code/sdi2100258/notebook52537b1dc1>
- DistilBERT notebook: <https://www.kaggle.com/code/sdi2100258/notebook2b31913fad/notebook>

2. Data Processing and Analysis

2.1. Pre-processing

Preprocessing is a critical step in any NLP pipeline, as it ensures that the data is clean and ready for feature extraction and modeling. For this task, we applied several preprocessing techniques to the raw text data. First, we unescaped HTML characters to handle encoded text properly and removed extra spaces to standardize the formatting.

We applied two preprocessing techniques. The first technique serves as a baseline, while the second is used to compare the performance of the classifiers after changing their embeddings. Finally, we converted all text to lowercase to ensure uniformity, as case differences (e.g., "Happy" vs. "happy") do not typically carry semantic meaning in this context and this is also automatically done by the models' tokenizers. The tokenizers truncate and pad tweets up to a maximum length that is decided for each preprocessing technique after statistical analysis.

The two preprocessing techniques can be summarized as follows:

- **Simple preprocessing:** all special characters are removed (including @ and #), along with links. Therefore, mentions and hashtags are treated as regular words.
- **Tag preprocessing:** special characters are maintained, while links, mentions and hashtags are tagged using < and >. These are added as new tokens to the models' tokenizers.

2.2. Analysis

Most of the analysis is skipped, as the dataset and EDA is identical to the first project and second projects. However, we do analyze the number of tokens after using the two models' tokenizers. This is done as it is necessary for the classifiers to have a uniform representation through padding and truncation, while it is also useful to pick a suitable maximum length that incorporates enough tweet tokens and maintains relatively low training times.

To identify a suitable maximum length, in Fig. 1 we plot the CDF of the token lengths on the training, validation and test sets that result from the tag preprocessing technique. Ideally, we want to pick the minimum value that guarantees that is enough for at least 99% of the tweets. Furthermore, we would like to avoid exceeding 64 tokens, as the training times become too long for higher token lengths. Based on these results, we use a maximum length of 59 tokens for the simple preprocessing technique as that covered 100% of the dataset and a maximum length of 64 tokens for the tag preprocessing technique.

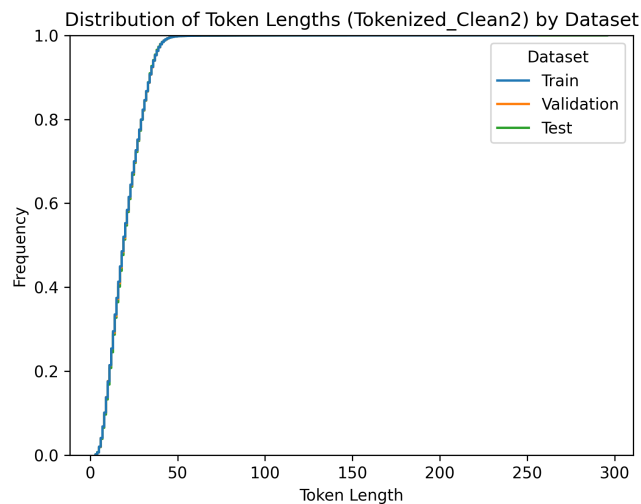


Figure 1: CDF of the token length across the different datasets resulting from the tag preprocessing technique.

2.3. Data Partitioning for Train, Test, and Validation

Since the dataset was already partitioned into training, validation, and test sets, we perform training on the training set and evaluation of the validation set. The test set is used only for the final Kaggle submission. Cross-validation is not used because of the lengthy training times of the neural networks.

2.4. Vectorization

For vectorization, we use the built-in models' tokenizers and embeddings, which map tokens to 768 dimensional vectors. In the case of the tag preprocessing technique, we also add the three special tokens to the embeddings, so this is increased to 771.

3. Algorithms and Experiments

3.1. Experiments

We use DistilBERT to guide our training as it is a smaller model so experimentation is much faster. The techniques that provided higher performance for DistilBERT are then also applied for BERT. The experiments are structured as follows:

- **Dummy:** a dummy classifier which predicts classes at random is used as a simple baseline model. All models should perform better than this or they do not have sufficient predictive power to be of practical use.
- **Model 1** the BERT and DistilBERT classifiers trained on the simple preprocessing technique with a batch size of 32. This is used as a simple model to improve upon.
- **Model 2** same as Model 1 except the tag preprocessing is used and the models' embeddings are resized to account for this.

The first set of experiments provided an indication that the tag preprocessing technique resulted in better performance. Therefore, this technique is used for all of the following experiments:

- **Model 3** same as Model 2 except a batch size of 16 is used.
- **Model 4** we fine-tune Model 3 using 10 Optuna trials to identify better learning rate, warm up rates and ϵ values for the optimizer and warm-up scheduler.

All experiments are trained using the AdamW optimizer with a learning rate of 10^{-5} . All models are trained for 2 epochs and the performance is measured across both training and validation datasets at the end of each epoch. More training epochs resulted in overfitting across all experiments so they are not included to standardize training times and save time. The reported scores are extracted at the end of the second epoch on the validation set.

3.1.1. Table of Trials. Table 1 shows the results for the different trials that took place for training and evaluating the DistilBERT models, while Table 2 shows the results for the BERT models. The precision, recall and F1-score correspond to the macro averages of the classification report returned by sklearn.

Model	Precision	Recall	F1-Score	Accuracy
Dummy	0.50	0.50	0.50	50%
Model 1	0.83	0.83	0.83	83%
Model 2	0.85	0.85	0.85	85%
Model 3	0.85	0.85	0.85	85%
Model 4	0.85	0.85	0.85	85%

Table 1: Trials with the DistilBERT models and their results on the validation set.

Model	Precision	Recall	F1-Score	Accuracy
Dummy	0.50	0.50	0.50	50%
Model 1	0.84	0.84	0.84	84%
Model 2	0.85	0.85	0.85	85%
Model 3	0.85	0.85	0.85	85%
Model 4	0.86	0.86	0.86	86%

Table 2: Trials for the BERT models and their results on the validation set.

3.2. Hyper-parameter Tuning

Regarding the hyper-parameter tuning performed using Optuna, the overall performance did not improve much throughout the trials. For the DistilBERT models, all trials resulted in performance between 84% and 85% validation accuracy after two epochs. The best hyperparameters were learning rate of $\approx 3.5 \cdot 10^{-5}$, $\epsilon \approx 4.2 \cdot 10^{-9}$ and warmup ratio of ≈ 0.087 . For the BERT models, all experiments achieved performance between 84% and 86%. The best hyperparameters found were a learning rate of $\approx 1.64 \cdot 10^{-5}$, a value of $\epsilon \approx 8.46 \cdot 10^{-9}$ and a warmup ratio of ≈ 0.034 .

3.3. Optimization techniques

The AdamW optimizer was used across all experiments with varying learning rates, epsilon and warm up rates.

3.4. Evaluation

Overall, the best performing model was Model 4, achieving the highest scores across all metrics for both DistilBERT and BERT. The tag preprocessing technique led to performance improvements for both types of models. The batch size decrease led to very little improvement, while fine-tuning the hyperparameters led to a notable performance increase.

The per-class scores did not significantly differ between most models. These scores are not displayed in the report, but they are evident in the confusion matrices shown Fig. 5 and Fig. 6. Very minor differences of approximately 1% are identified between classes. For instance, Models 2 and 3 of BERT achieve 85% precision for the negative class and 86% for the positive class, while the scores are reversed for recall (86% for the negative and 85% for the positive class). This indicates that the model is slightly more likely to predict the negative class, leading to an increase in recall but a decrease in precision. This is not as prominent in Model 4. For DistilBERT, the scores are balanced across both classes.

All in all, the input and training hyperparameters seem to be the most important factor in model performance, as the different models exhibit more or less equal performance. BERT also always outperforms DistilBERT at the cost of much higher training times. Perhaps a better preprocessing technique would have further increased scores past the approximately 84% to 85% scores shown here. However, the training times were too long to facilitate further experimentation.

3.4.1. ROC curve. The ROC curves of all DistilBERT and BERT models are depicted in Fig. 2.

Most of the DistilBERT models have practically identical performance, except Model 1 trained on the simple preprocessing technique, which performs slightly worse. Model 1 has the worst AUC at 0.93. Model 3 seems to perform very slightly better than Model 2 but the increase is practically negligible, with both achieving AUC of 0.92. Model 4 has the highest AUC at 0.94 as it slightly outperforms the rest of the models.

A very similar behavior can be observed for the BERT models, except they achieve slightly higher scores, with Model 1 achieving 0.92 AUC. Overall, Model 4 seems to perform the best, achieving an AUC of 0.93.

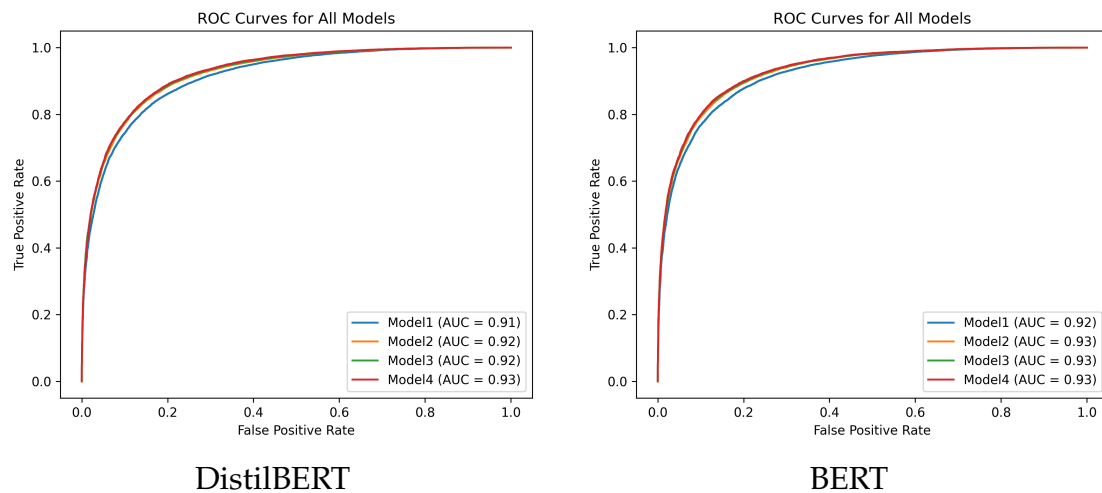


Figure 2: ROC curves of all DistilBERT and BERT models.

3.4.2. Learning Curve. The learning curve for each DistilBERT and BERT model is depicted in Fig. 3. Detailed comments about each model for both DistilBERT and BERT:

- Model 1: This model demonstrates consistent performance with both training and validation accuracy starting high and improving slightly, while both losses steadily decrease. It shows good generalization as validation metrics closely track training metrics throughout the initial two epochs.
- Model 2: exhibits very similar trends to Model 1, this model also starts with high accuracy and low loss, both of which improve slightly over the two epochs. Its strong alignment between training and validation curves indicates good generalization from the outset.
- Model 3: While validation accuracy initially lags slightly, it quickly catches up to and closely follows the training accuracy, both showing positive improvement over the two epochs. This model also demonstrates solid generalization with decreasing losses for both training and validation.
- Model 4: This model stands out by achieving the highest initial and final accuracy among the four within the two epochs, with both training and validation accuracy showing a robust increase. It maintains excellent generalization, as its validation metrics closely mirror its training metrics, suggesting the most promising performance. This model does exhibit a slightly higher gap between training and validation performance, which might indicate very slight overfitting.

3.4.3. Confusion matrix. The confusion matrix extracted from each DistilBERT model for the validation set predictions is depicted in Fig. 5 while for the BERT models they are depicted in Fig. 6.

The confusion matrices of DistilBERT do exhibit very slight differences between models. Model 2 seems to produce more False Negatives (FN) compared to other models, but Models 3 and 4 trained on the same preprocessing technique do not suffer from this. The overall best results seem to be achieved from Model 4, achieving the highest number of True Positives (TP) but slightly fewer True Negatives (TN) than Model 3.

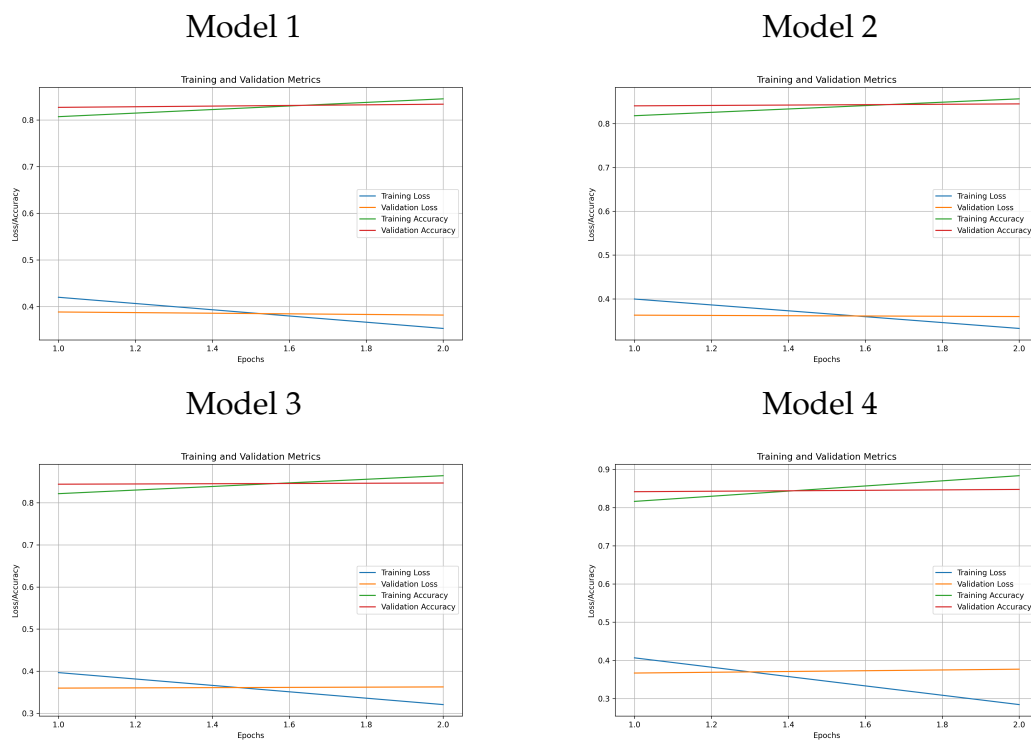


Figure 3: Learning curves for all DistilBERT models.

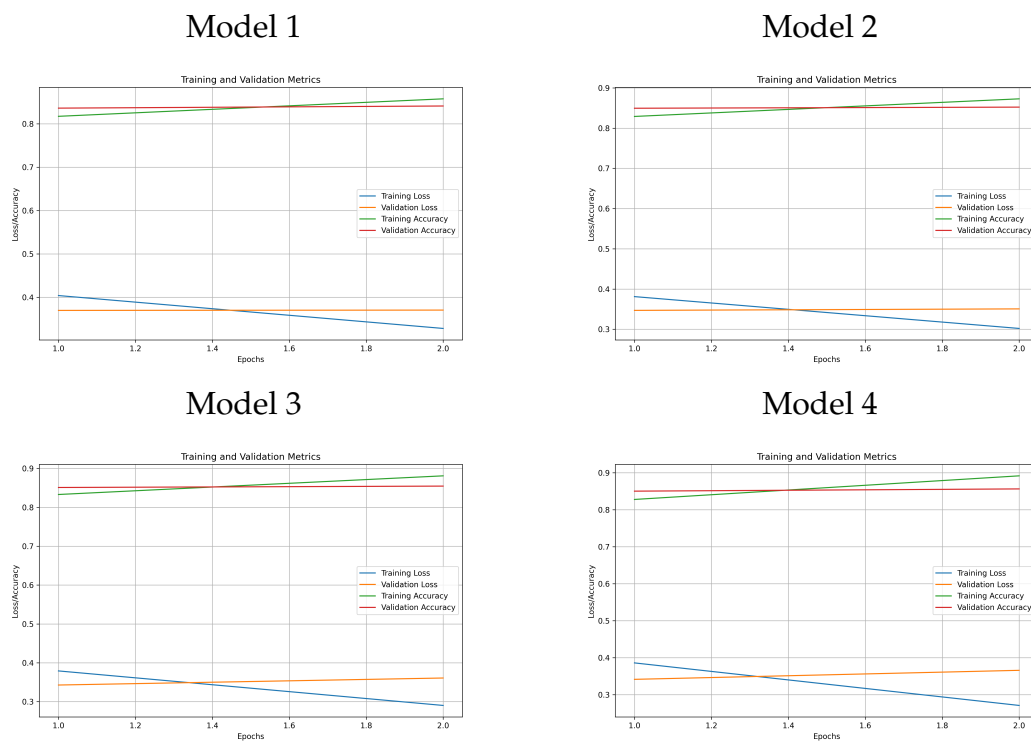


Figure 4: Learning curves for all BERT models.

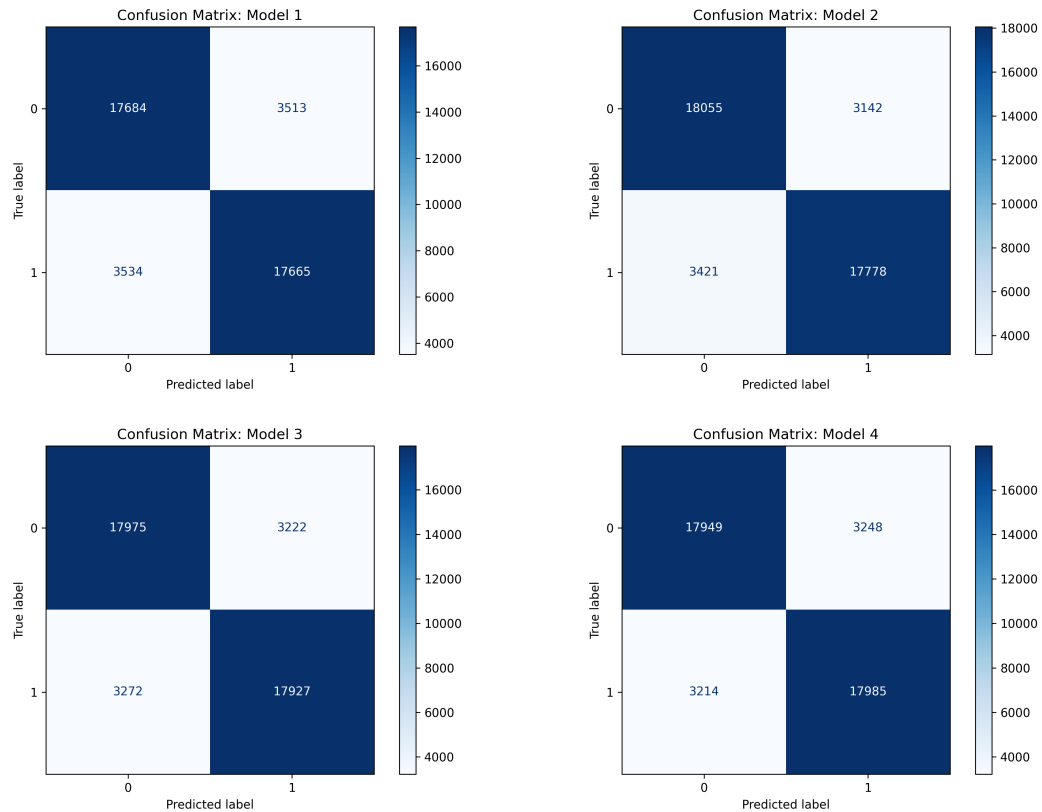


Figure 5: Confusion matrices for all DistilBERT models.

This is slightly more exaggerated for the BERT models, as most experiments seem to produce more FN than False Positives (FP). Model 4 performs the best, having a relatively balanced distribution across FN and FP.

4. Results and Overall Analysis

4.1. Results Analysis

The results acquired from our approach are overall good, as we achieved an accuracy score of over 84% on the validation set with our best models. This was a significant increase compared to the dummy classifier. Slight performance increases were obtained from improved preprocessing and hyperparameter fine-tuning.

4.2. Comparison with the first project

When comparing performance to the first model, it seems that the achieved scores are higher across the board. The much more powerful BERT-based models were able to improve upon the performance of TF-IDF and Logistic Regression, although at the cost of much more extensive training times. However, it could be claimed that the performance increase is not significant enough to warrant the much more computationally expensive models, but this depends on the context of the task. Nonetheless, it is impressive that a much more basic approach is relatively comparable to the BERT-based models.

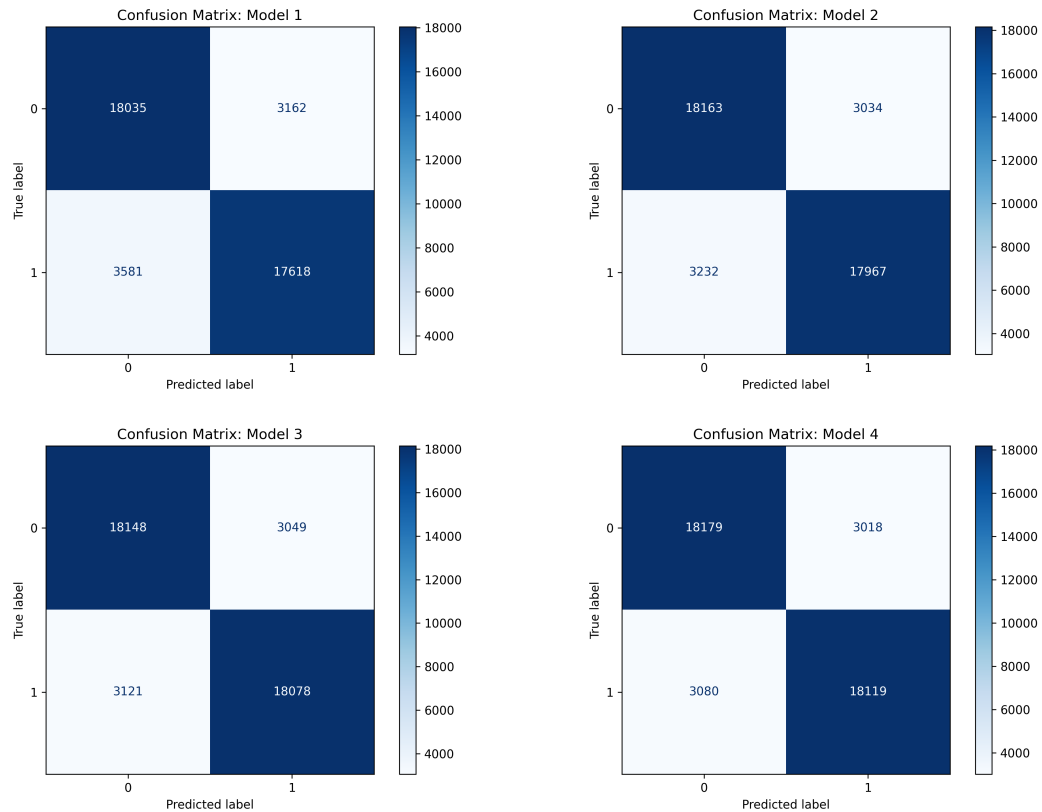


Figure 6: Confusion matrices for all BERT models.

4.3. Comparison with the second project

Compared to the second project, the performance gains are even more impressive. Despite the MLP models also utilizing embeddings to represent text, the BERT-based models achieve much better performance. This is likely due to the models having much higher embedding dimensions as well as a much more sophisticated architecture compared to the simple MLP models we designed. This still results in much longer training times, but also much better performance. Furthermore, transfer learning from more general tasks to more specific tasks is known to improve performance, so the higher scores were expected.

5. Bibliography

References

- [1] Daniel Jurafsky and James H. Martin. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd edition, 2025. Online manuscript released January 12, 2025.